



AHMEDABAD INSTITUTE OF TECHNOLOGY

Department of Computer Engineering

(Affiliated to Gujarat Technological University)

A Project Report on

AQI ANALYSIS AND PREDICTION DASHBOARD USING MACHINE LEARNING

Problem Based Learning (PBL) Activity — Project Management (BE05000481)

B.E. Semester 5 — Computer Engineering

Academic Year 2026–27

Submitted by

Jay Patel — 240020107079

Savan Patel — 240020107091

Guided by

Dr. Apeksha Dave

Subject Faculty & Subject Co-ordinator

Dr. Harsha Padheriya

Head of Department, Computer Engineering

Acknowledgement

We would like to express our sincere gratitude to Dr. Apeksha Dave, Subject Faculty and Subject Co-ordinator for Project Management (BE05000481), for her guidance throughout this Problem Based Learning activity and for her feedback on structuring the planning documentation. We are also grateful to Dr. Harsha Padheriya, Head of the Department of Computer Engineering, and to Ahmedabad Institute of Technology for the infrastructure and learning environment that made this project possible. We would also like to thank our classmates for their support and useful discussion during the development and testing of the dashboard.

Abstract

Air pollution is one of the most pressing environmental health concerns in Indian cities, and the Air Quality Index (AQI) published by the Central Pollution Control Board is the primary way this risk is communicated to the public. This project builds a machine-learning system that predicts AQI directly from pollutant concentrations and presents it through an interactive Streamlit dashboard, removing the need for a user to work through CPCB's sub-index calculations by hand.

Using the "Air Quality Data in India (2015–2020)" dataset — 29,531 daily records across 26 cities and twelve pollutant parameters — four regression models were trained and compared: Linear Regression, Decision Tree, Random Forest, and Gradient Boosting. Random Forest gave the best fit ($R^2 = 0.911$, MAE = 20.50 on a held-out 4,970-record test set) and was refined further with a grid search over 24 hyperparameter combinations under 5-fold cross-validation. The tuned model, along with a city-encoding table, was packaged into a Streamlit application: a user selects a city, enters pollutant readings, and receives the predicted AQI, its CPCB severity category, and pollutant-specific health advice in real time.

Beyond the technical build, this report documents the project through a project-management lens — work breakdown, Gantt and PERT/CPM scheduling, budget and risk analysis — and closes with the system's design, results, and scope for future work.

Table of Contents

Acknowledgement **2**
Abstract **3**
Table of Contents **4**
List of Figures **5**
List of Tables **5**
List of Abbreviations **5**
Chapter 1: Introduction **6**
1.1 Overview 6
1.2 Project Background 6
1.3 Problem Statement 6
1.4 Need for the Project 6
1.5 Objectives 7
1.6 Scope of the Project 7
1.7 Project Limitations 7
Chapter 2: Project Planning **8**
2.1 Project Management Methodology (Waterfall Model) 8
2.2 Project Plan 8
2.3 Project Life Cycle 8
2.4 Work Breakdown Structure (WBS) 9
2.5 Gantt Chart 9
2.6 PERT/CPM Network Diagram 10
2.7 Budget Estimation 10
2.8 Resource Allocation 10
2.9 Risk Analysis 11
2.10 Stakeholder Analysis 11
Chapter 3: System Design **12**
3.1 System Architecture 12
3.2 Use Case Diagram 13
3.3 Entity Relationship (ER) Diagram 13
3.4 Communication Plan 14
3.5 Quality Management Plan 14
3.6 Project Monitoring and Control 14
Chapter 4: Results and Discussion **15**
4.1 Expected Results 15
4.2 Advantages 16
4.3 Limitations 16
4.4 Future Scope 16
4.5 Conclusion 16
References **17**
Appendices **17**

List of Figures

Fig. 2.1 — Work Breakdown Structure (WBS)
Fig. 2.2 — Gantt Chart
Fig. 2.3 — PERT / CPM Network Diagram
Fig. 3.1 — System Architecture
Fig. 3.2 — Use Case Diagram
Fig. 3.3 — Entity Relationship (ER) Diagram
Fig. 4.1 — Pollutant Correlation with AQI
Fig. 4.2 — Model Performance Comparison

List of Tables

Table 2.1 — Budget Estimation
Table 2.2 — Resource Allocation
Table 2.3 — Risk Analysis
Table 2.4 — Stakeholder Analysis
Table 3.1 — Communication Plan
Table 4.1 — Model Comparison Results

List of Abbreviations

AQI	Air Quality Index	**CPCB**	Central Pollution Control Board
ML	Machine Learning	**UI**	User Interface
WBS	Work Breakdown Structure	**PERT**	Program Evaluation & Review Technique
CPM	Critical Path Method	**ER**	Entity Relationship
MAE	Mean Absolute Error	**RMSE**	Root Mean Squared Error
R^2	Coefficient of Determination	**PM2.5 / PM10**	Particulate Matter (≤2.5 / ≤10 microns)
NOx / NH3	Nitrogen Oxides / Ammonia	**CO / SO2 / O3**	Carbon Monoxide / Sulphur Dioxide / Ozone
CSV	Comma-Separated Values	**PBL**	Problem Based Learning
GTU	Gujarat Technological University	**CV**	Cross-Validation

Chapter 1: Introduction

1.1 Overview

Rising vehicular emissions, construction dust, and industrial output have pushed several Indian cities into the “Poor” to “Severe” Air Quality Index (AQI) bands for extended parts of the year. AQI compresses eight pollutant sub-indices into one number the public can act on, but computing it from raw sensor readings requires knowing the Central Pollution Control Board's (CPCB) piecewise breakpoint tables for each pollutant — not something a resident checking today's air quality can do by hand. This project trains a regression model on historical pollutant data to approximate that mapping and wraps it in a dashboard, so a user can enter pollutant values and get back an AQI reading, its category, and practical health advice.

1.2 Project Background

The project uses the “Air Quality Data in India (2015–2020)” dataset (city-day granularity), which aggregates CPCB monitoring-station readings for 26 cities — including Delhi, Mumbai, Bengaluru, Ahmedabad, and Chennai — into twelve pollutant columns (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene) alongside the recorded AQI and its bucket label. At 29,531 rows spanning five years, it is large enough to support supervised regression while staying small enough to train and tune entirely on a personal laptop — well suited to the scope of a semester micro-project.

1.3 Problem Statement

Given historical, city-wise pollutant concentration readings, can a regression model predict the Air Quality Index accurately enough (target: $R^2 > 0.85$ on unseen data) to serve as the basis for a real-time advisory dashboard, and can that prediction be delivered through an interface simple enough for a non-technical user?

1.4 Need for the Project

Official AQI figures are published city-wide on a fixed update schedule; they do not let a resident ask “what would the AQI be for these specific readings” or get instant, plain-language health guidance attached to the number. A lightweight predictive tool fills that

gap — useful here for demonstrating the ML pipeline end to end, and, with live data feeds, extensible into an actual public-facing advisory tool.

1.5 Objectives

- Collect and preprocess multi-city, multi-year pollutant data into a form suitable for regression modelling.
- Compare multiple regression algorithms (Linear Regression, Decision Tree, Random Forest, Gradient Boosting) and select the best-performing one.
- Tune the selected model's hyperparameters via grid search to improve on the untuned baseline.
- Build an interactive Streamlit dashboard that returns AQI, its category, and pollutant-specific health advice from user-entered values.
- Evaluate system reliability quantitatively using MAE, RMSE, and R^2 on held-out test data.

1.6 Scope of the Project

The system covers the 26 cities and twelve pollutants present in the source dataset (2015–2020). It performs point prediction of AQI from manually entered pollutant values through a local Streamlit interface; it does not, in this version, pull live sensor data or expose a public API.

1.7 Project Limitations

- The model is trained on 2015–2020 data and will not reflect emission or policy changes after that window unless retrained on newer data.
- There is no live sensor or weather-API integration — every prediction depends on values the user enters manually.
- Several pollutant columns had substantial missing data before cleaning (PM10 missing in 24% of rows, Xylene in 52%); these were median-imputed, a reasonable default that can still understate genuinely extreme pollution events.
- Hyperparameter tuning improved the Random Forest only marginally over its untuned baseline (Section 4.1) — the practical accuracy ceiling appears to be set by data quality rather than model choice.

Chapter 2: Project Planning

2.1 Project Management Methodology (Waterfall Model)

This project followed a Waterfall structure, appropriate for a fixed-scope micro-project where each phase's output is a hard prerequisite for the next: models cannot be trained before the data is cleaned, and the dashboard cannot be meaningfully built before a trained model exists to call. The five phases — Project Planning, Data Engineering, Model Development, Application Development, and Testing & Deployment — were executed in sequence, shown in the Work Breakdown Structure (Fig. 2.1). The one departure from strict Waterfall was inside Data Engineering: feature correlations were computed once before dropping missing-AQI rows and recomputed after, a short, contained iteration rather than a return to an earlier phase.

2.2 Project Plan

The project was planned for two team members over 15 working days (three weeks), with Dashboard Development running in parallel to Model Development to shorten the overall schedule. The day-by-day plan is shown in the Gantt chart (Fig. 2.2), and task dependencies are shown in the PERT/CPM network (Fig. 2.3).

2.3 Project Life Cycle

- Planning — define the problem statement and objectives, identify the dataset.
- Data Engineering — clean, impute, and explore the data; select features.
- Model Development — train, evaluate, and tune candidate regression models.
- Application Development — build the Streamlit UI and wire it to the trained model.
- Testing & Deployment — test end to end and finalise documentation.

2.4 Work Breakdown Structure (WBS)

Fig. 2.1 — Work Breakdown Structure (WBS)

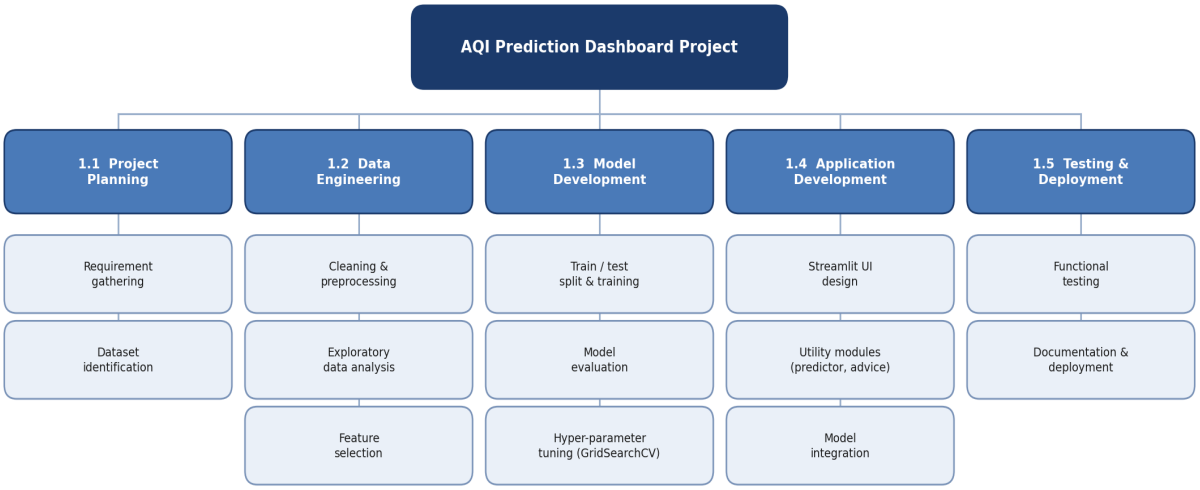


Fig. 2.1 — Work Breakdown Structure (WBS)

Fig. 2.1 decomposes the project into the five Level-1 phases above and twelve Level-2 tasks. Data Engineering and Model Development carry the most granular breakdown because they involved the most decision points — which imputation strategy to use, which four algorithms to compare, and which hyperparameters to search over.

2.5 Gantt Chart

Fig. 2.2 — Gantt Chart (15 Working-Day Schedule)

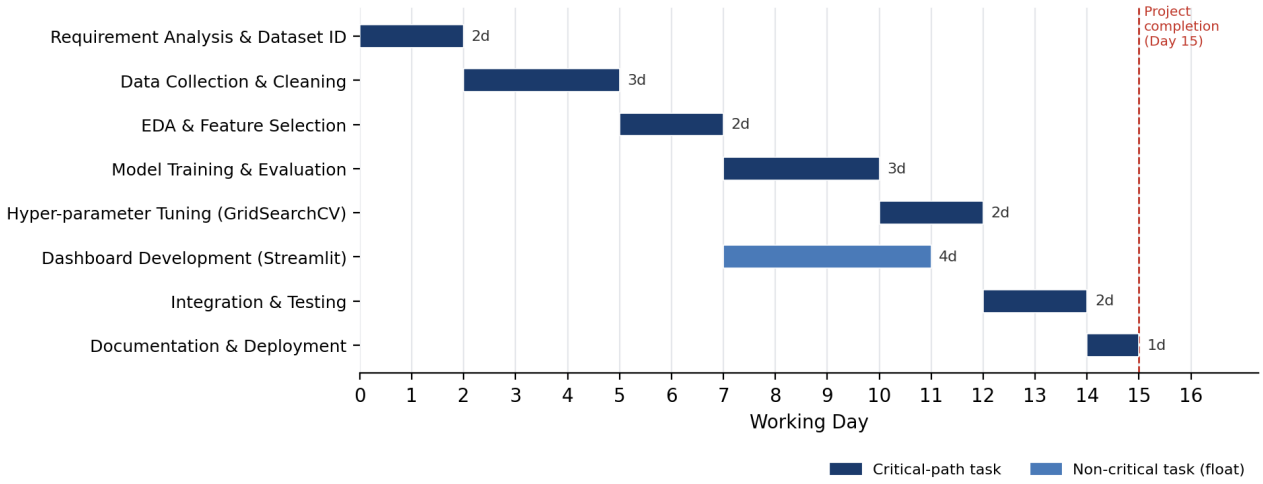


Fig. 2.2 — Gantt Chart (15 working-day schedule)

Fig. 2.2 schedules the twelve tasks across 15 working days. Dashboard Development (4 days) starts as soon as feature selection finishes and runs alongside Model Training,

Evaluation, and Tuning (5 days combined); since neither branch depends on the other's output until Integration & Testing, running them in parallel saves roughly four days against a fully sequential plan.

2.6 PERT/CPM Network Diagram

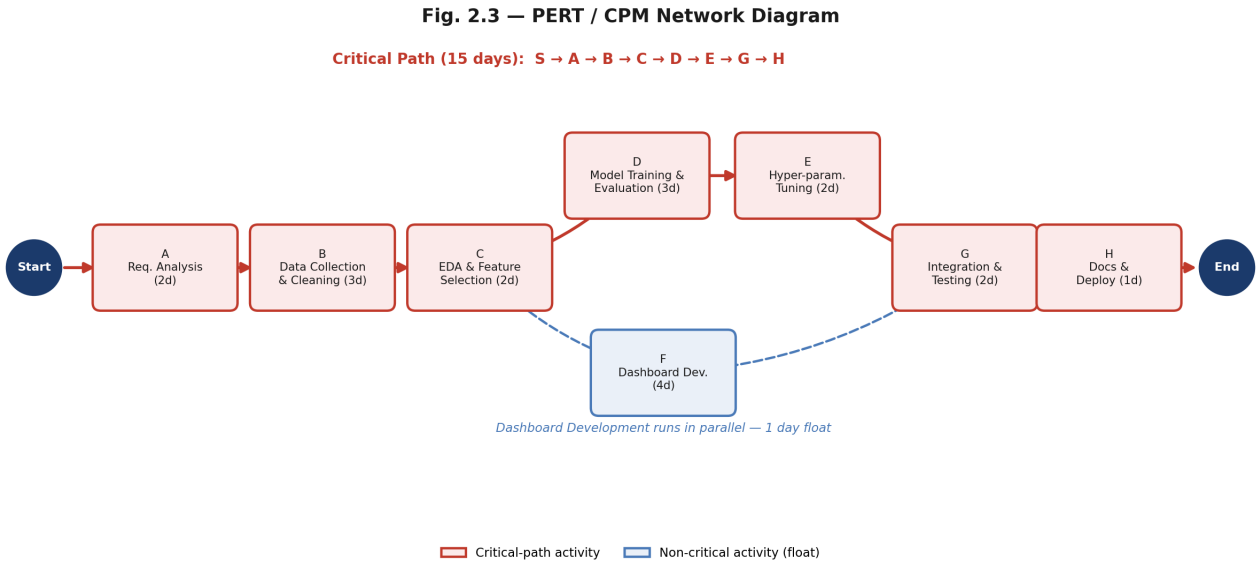


Fig. 2.3 — PERT / CPM Network Diagram

The critical path — Requirement Analysis → Data Collection & Cleaning → EDA & Feature Selection → Model Training & Evaluation → Hyperparameter Tuning → Integration & Testing → Documentation & Deployment — totals 15 days and sets the project's minimum duration. Dashboard Development is the only non-critical activity, carrying one day of float.

2.7 Budget Estimation

Every library used — pandas, scikit-learn, Streamlit, joblib — is open-source, and the dataset is freely available on Kaggle, so the project's actual cash outlay was close to zero. Table 2.1 instead costs the project notionally by effort, the more meaningful figure for a two-person academic build.

Table 2.1 — Budget Estimation

Item	Basis	Estimated Cost (₹)
Software & libraries (Python, scikit-learn, Streamlit, pandas)	Open-source	0

Item	Basis	Estimated Cost (₹)
Dataset (Kaggle: Air Quality Data in India)	Public / free	0
Development effort	2 members × ~60 h × ₹150/h (notional)	18,000
Cloud hosting / deployment	Streamlit Community Cloud, free tier	0
Documentation & testing	Notional	1,000
Total		₹19,000

2.8 Resource Allocation

Table 2.2 — Resource Allocation

Resource	Type	Allocation
Team members (2)	Human	Data engineering, model development, app development, and testing (shared)
Faculty guide	Human	Weekly review, milestone sign-off
Personal laptop	Hardware	Development & model training (CPU only — dataset small enough to not need GPU)
Python 3, scikit-learn, pandas, Streamlit	Software	Full stack, all phases
Kaggle dataset	Data	Model training & evaluation

2.9 Risk Analysis

Table 2.3 — Risk Analysis

Risk	Likelihood	Impact	Mitigation
Missing / inconsistent pollutant data	Medium	High	Median imputation after auditing null counts per column (Sec. 1.7)
Model overfitting	Medium	Medium	80/20 train-test split; 5-fold cross-validation during tuning
Low prediction accuracy	Low	High	Compared 4 algorithms; selected Random Forest (highest R^2)
Deployment / environment mismatch	Medium	Low	requirements.txt pinned; tested locally before packaging

Risk	Likelihood	Impact	Mitigation
Schedule slippage	Medium	Medium	Parallel scheduling of Dashboard Dev. against Model Dev. (Fig. 2.3)

2.10 Stakeholder Analysis

Table 2.4 — Stakeholder Analysis

Stakeholder	Interest	Influence
General public / end users	Quick, understandable air-quality reading and health guidance	Medium
Project team	Deliver a working, accurately evaluated system	High
Faculty evaluator	Assess technical understanding, novelty, and report quality	High
Public-health / environmental bodies (indirect)	Broader awareness tools for air-quality risk	Low

Chapter 3: System Design

3.1 System Architecture

Fig. 3.1 — System Architecture

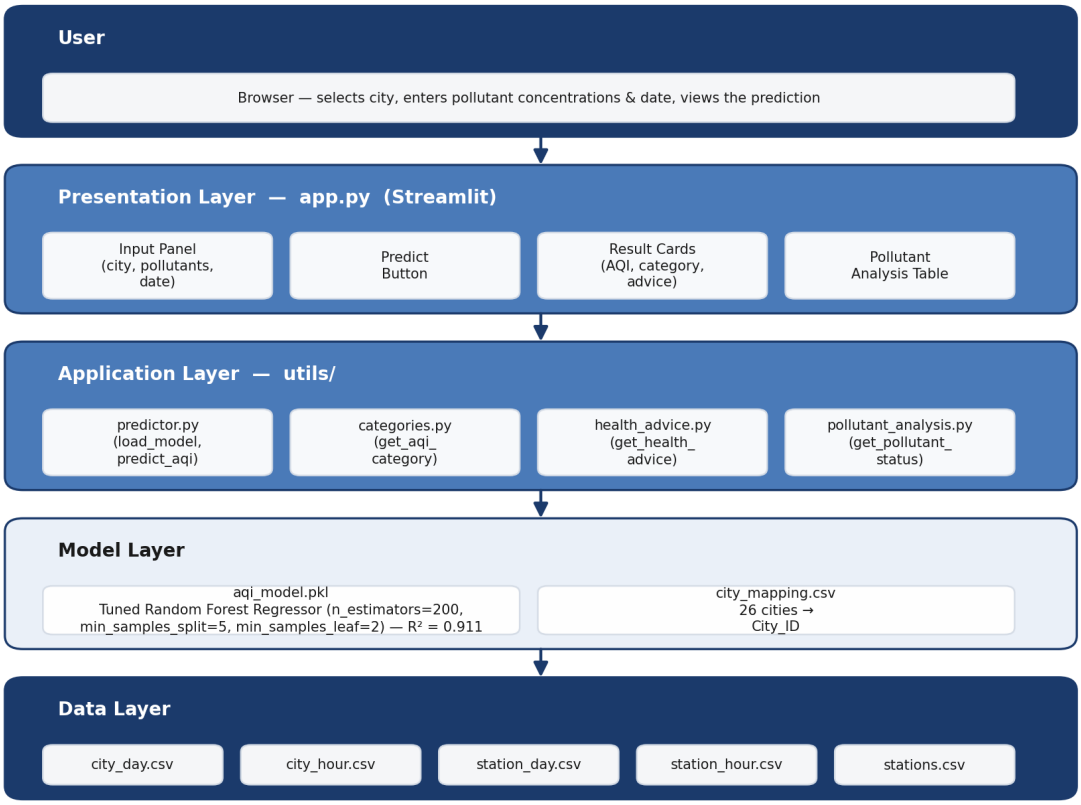


Fig. 3.1 — System Architecture

The system is organised into five layers (Fig. 3.1). The user interacts with the Presentation Layer (app.py), a Streamlit interface that collects city, pollutant, and date inputs and renders results. It calls four modules in the Application Layer (utils/) that separate concerns cleanly: predictor.py loads the serialized model and runs inference, categories.py maps a numeric AQI to its CPCB bucket, health_advice.py returns bucket-specific guidance, and pollutant_analysis.py flags individual pollutant values as normal or elevated. The Model Layer holds the tuned Random Forest (aqi_model.pkl) and the city-to-ID encoding (city_mapping.csv) that predictor.py depends on. The Data Layer — five CSV files supplied with the original dataset — is used at training time and is not queried at inference time, since the trained model already encodes what it learned from that data.

3.2 Use Case Diagram

Fig. 3.2 — Use Case Diagram

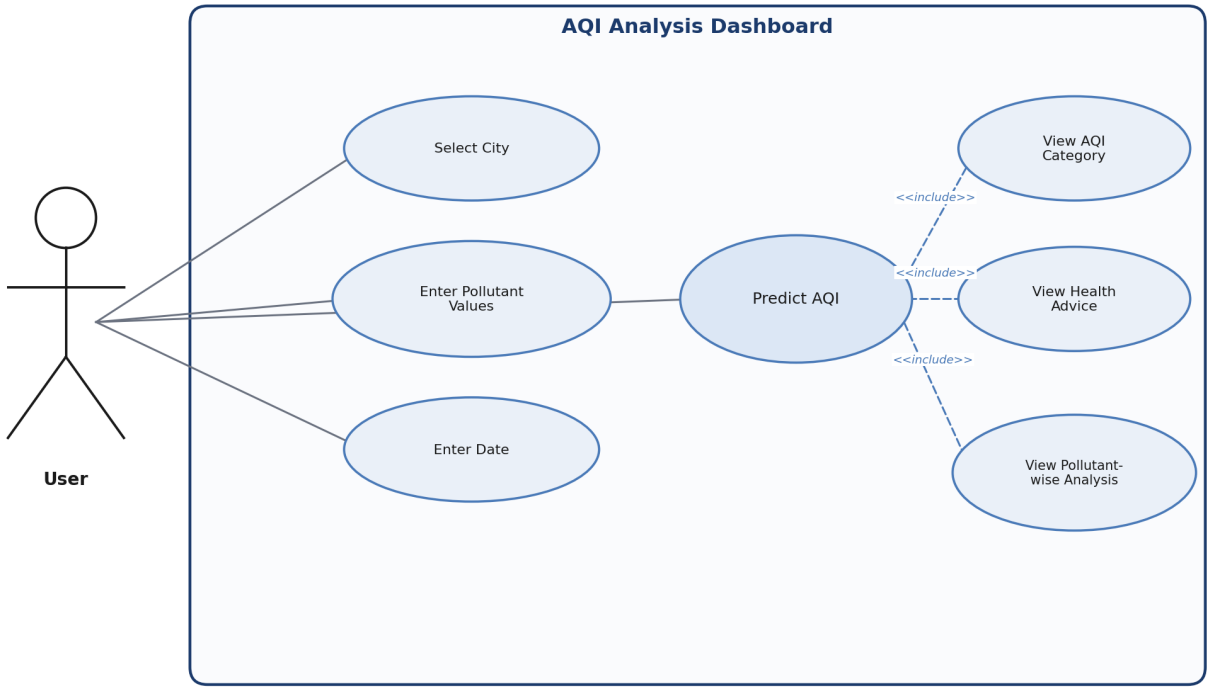
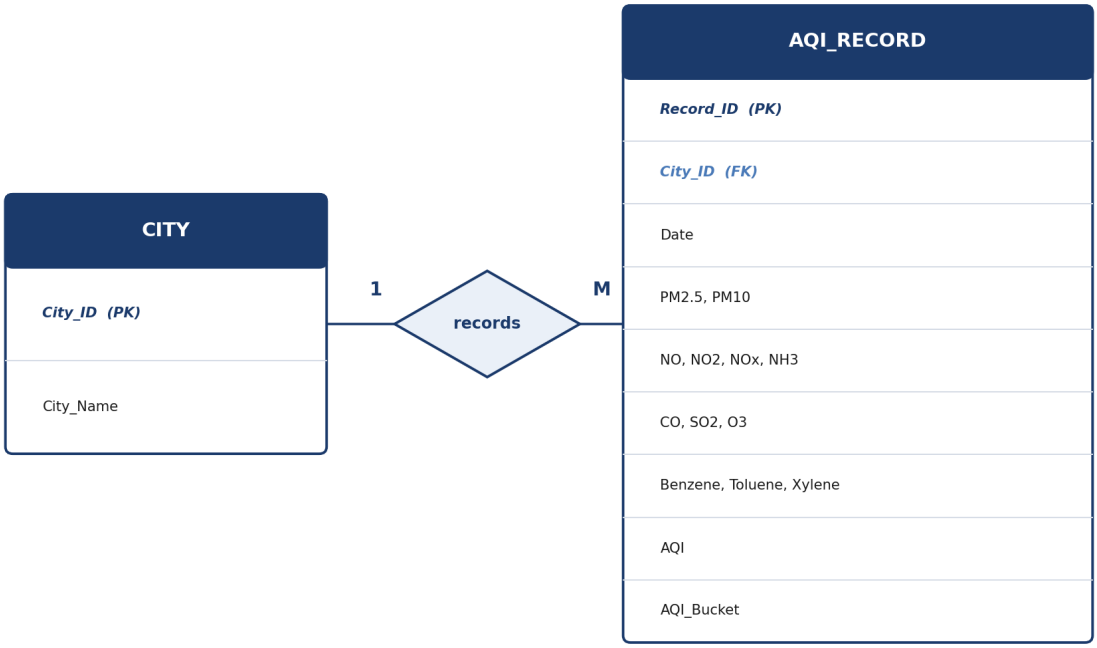


Fig. 3.2 — Use Case Diagram

As Fig. 3.2 shows, the single actor — the User — directly triggers four use cases: selecting a city, entering pollutant values, entering a date, and predicting AQI. Predicting AQI includes three further use cases (View AQI Category, View Health Advice, View Pollutant-wise Analysis) that always fire together once a prediction is made, which is why they are modelled with <<include>> rather than as actions the user triggers independently.

3.3 Entity Relationship (ER) Diagram

Fig. 3.3 — Entity Relationship (ER) Diagram



One CITY has many AQI_RECORD entries (daily pollutant readings); City_ID is the foreign key linking them.

Fig. 3.3 — Entity Relationship (ER) Diagram

The current implementation reads directly from flat CSV files rather than a relational database, but Fig. 3.3 documents the conceptual schema those files represent: a CITY entity (City_ID, City_Name) in a one-to-many relationship with AQI_RECORD (date-stamped pollutant readings and the resulting AQI/bucket), linked by City_ID as a foreign key. This is the schema a database-backed version of the dashboard — one of the future-scope items in Section 4.4 — would implement directly.

3.4 Communication Plan

Table 3.1 — Communication Plan

Stakeholder	Frequency	Mode	Purpose
Team members	Daily (active phases)	In-person / chat	Progress sync, unblock issues
Faculty guide	Weekly	In-person review	Milestone review, feedback
End users	Post-deployment	Dashboard interface	Implicit feedback via usage

3.5 Quality Management Plan

Quality was checked at two levels. Quantitatively, every candidate model was scored on the same held-out test set using MAE, MSE, RMSE, and R^2 (Table 4.1) before Random Forest was selected and tuned — a model was only carried forward if it beat the previous best on R^2. Functionally, the Streamlit UI was tested manually across the input range, including zero-value pollutant entries (the app's default), to confirm predictions and health advice render without errors, and the two team members cross-checked each other's utils/ modules before integration.

3.6 Project Monitoring and Control

Progress was tracked against the WBS task list and the Gantt schedule (Figs. 2.1–2.2): each completed task was checked off against its planned day, and the two tasks running in parallel — Model Development and Dashboard Development — were compared at the point they merged into Integration & Testing to confirm neither had slipped past the other. Any deviation from the critical path in Fig. 2.3 would have been the trigger to re-plan; in practice the project stayed on schedule.

Chapter 4: Results and Discussion

4.1 Expected Results

Table 4.1 — Model Comparison Results (test set, n = 4,970)

Model	MAE	RMSE	R² Score
Linear Regression	30.99	59.27	0.808
Decision Tree	28.29	55.99	0.829
Random Forest	20.50	40.38	0.911
Gradient Boosting	23.39	43.24	0.898

Fig. 4.1 — Pollutant Correlation with AQI

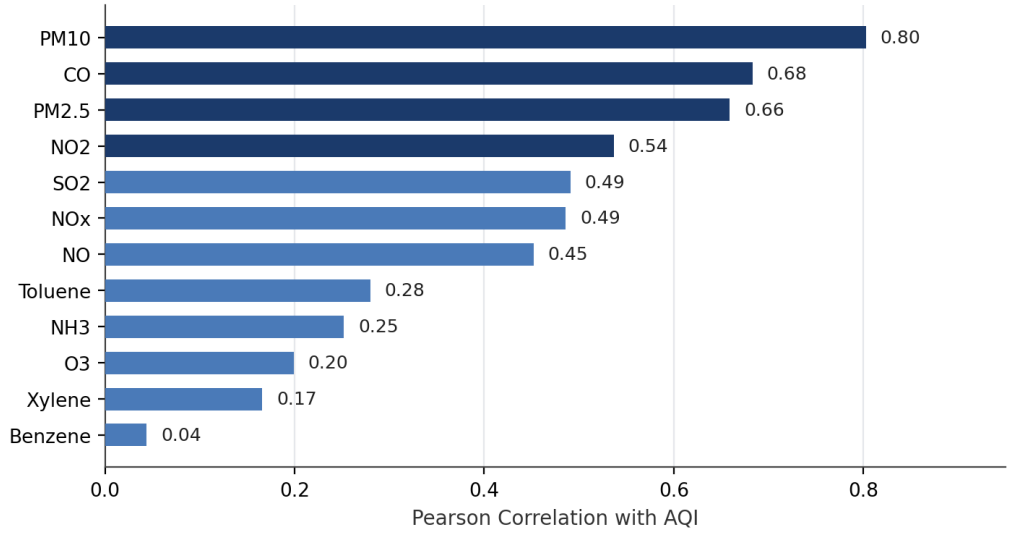


Fig. 4.1 — Pollutant Correlation with AQI

Fig. 4.2 — Model Performance Comparison (Test Set, n = 4,970)

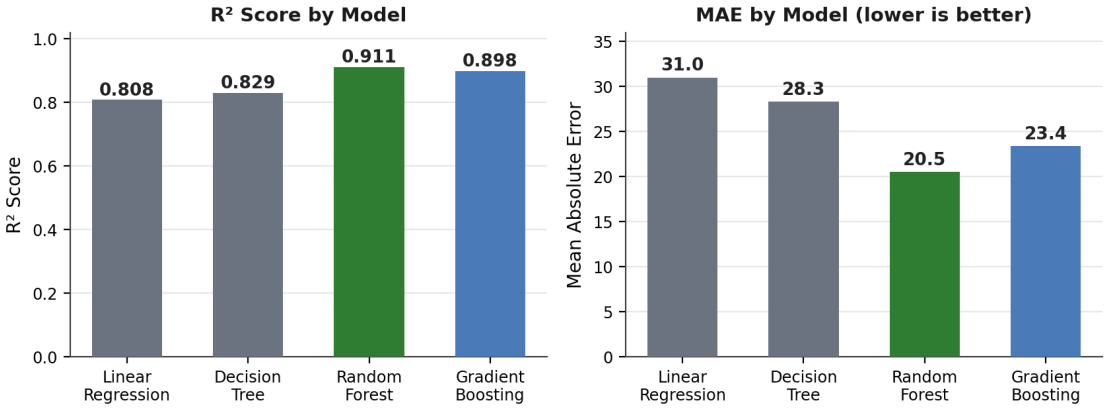


Fig. 4.2 — Model Performance Comparison

Random Forest was the clear winner among the four (Fig. 4.2), and Fig. 4.1 helps explain why: PM10, CO, and PM2.5 are the pollutants most correlated with AQI (r = 0.80, 0.68, 0.66), and a tree-ensemble model captures the non-linear, threshold-like way these particulates drive AQI better than a linear model can. Grid search over 24 hyperparameter combinations (5-fold CV) selected n_estimators=200, min_samples_split=5, min_samples_leaf=2 as the best configuration — but on the held-out test set this tuned model scored R^2 = 0.9110 against the untuned baseline's 0.9109, and MAE improved only from 20.499 to 20.442. The honest read is that the default Random Forest was already close to this dataset's ceiling; the missing-value imputation described in Section 1.7 likely bounds accuracy more than hyperparameter choice does at this point.

4.2 Advantages

- Turns a technical, multi-pollutant computation into a single number and a plain-language recommendation.
- Compares four algorithms rather than committing to one, with the comparison documented and reproducible.
- Runs entirely on open-source tools with no licensing cost.
- Modular utils/ structure (Section 3.1) makes it straightforward to swap in a new model or add a pollutant without touching the UI code.

4.3 Limitations

- Evaluated only on the 26 cities in the training data; predictions for other locations would be extrapolation.
- The dashboard reports a point estimate with no confidence interval, so a user cannot tell how uncertain a given prediction is.
- Depends on the user entering pollutant values accurately — there is no input validation against physically plausible ranges yet.

4.4 Future Scope

Several extensions were identified but are out of scope for this submission: live AQI / weather-API integration in place of manual entry, historical trend visualisation and short-term AQI forecasting, a downloadable PDF report of results, interactive charts within the dashboard itself, and user authentication for saved history.

4.5 Conclusion

The project met the objectives set out in Section 1.5: pollutant data for 26 cities was cleaned and explored, four regression models were compared on identical test data, Random Forest was selected and tuned, and the result was packaged into a Streamlit dashboard that returns an AQI prediction, category, and health advice in real time. At $R^2 = 0.911$, the model explains the large majority of variance in AQI from pollutant readings alone, with the remaining gap attributable more to missing-data imputation than to model choice — a reasonable outcome for a dataset assembled from real monitoring stations rather than a clean benchmark set.

References

1. Central Pollution Control Board (CPCB), National Air Quality Index. https://cpcb.nic.in
2. Kaggle: “Air Quality Data in India (2015–2020)” dataset.
3. Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python.” scikit-learn documentation, https://scikit-learn.org
4. Streamlit Inc., Streamlit Documentation. https://docs.streamlit.io
5. McKinney, W., “Data Structures for Statistical Computing in Python.” pandas documentation, https://pandas.pydata.org

Appendices

- Appendix A: Work Breakdown Structure — see Fig. 2.1, Section 2.4.
- Appendix B: Gantt Chart — see Fig. 2.2, Section 2.5.
- Appendix C: PERT/CPM Network Diagram — see Fig. 2.3, Section 2.6.